

ESTUDIO — Carpeta config/

Material de estudio para el final del TFC Bajoneá. Explicación en criollo, orientada a poder contarlo en una mesa de examen. Basado en el código real de `backend/src/main/java/com/bajonea/backend/config/` .

De qué va toda esta carpeta

`config/` es donde vive la **configuración de la aplicación**: cosas que no son ni pantallas, ni reglas de negocio, ni base de datos. Son "los preparativos" que Spring hace **una sola vez, al arrancar**, para que después todo lo demás funcione.

Hay dos grupos:

- `config/` a secas — 3 clases que le dicen a Spring "creá este objeto una vez y guardalo para que cualquiera lo pida": el cliente de Cloudinary, el cliente de Resend (mails) y la config de Swagger.
- `config/security/` — 7 clases. Acá está **todo el sistema de seguridad**: el JWT, quién entra a qué, qué pasa cuando alguien no tiene permiso. **Esto es lo más preguntable de la mesa**, prestale atención a esta parte.

Un concepto que se repite y conviene tener claro antes de seguir: un `@Bean` es un objeto que Spring crea al arrancar y después **inyecta** (le pasa solo) a cualquier clase que lo pida en su constructor. Ejemplo: `CloudinaryConfig` crea el objeto `cloudinary` una vez; después `CloudinaryService` simplemente lo pide y Spring se lo da ya armado. No tenés que hacer `new` nunca.

PARTE 1 — config/ (las 3 de configuración general)

1.1 CloudinaryConfig.java

Para qué sirve

Cloudinary es el servicio externo donde se guardan **todas las imágenes** del proyecto (fotos de productos, foto de perfil del comercio, foto del usuario). Esta clase arma **el cliente** para hablar con Cloudinary, con las credenciales cargadas.

Qué hace por dentro

Tiene un solo método:

Método	Qué hace
<code>cloudinary(cloudName, apiKey, apiSecret)</code>	Devuelve un objeto <code>cloudinary</code> configurado con las 3 credenciales, y con <code>secure: true</code> (o sea, siempre HTTPS).

Los 3 parámetros llegan con `@Value("${cloudinary.cloud-name}")` , etc. Eso significa: "**leelo del archivo `application.properties`** ", que a su vez lo lee de una variable de entorno. Traducido: las credenciales reales **nunca están escritas en el código**, es una regla del proyecto (CLAUDE.md, regla 6).

Cómo se conecta con el resto

Lo usa `CloudinaryService` , que es quien genera las "firmas" de subida. Concepto clave para la mesa: **el backend nunca recibe la imagen**. El frontend le pide al backend una firma, el backend se la da, y el frontend sube la foto **directo a Cloudinary**. Después el frontend le manda al backend solo la URL resultante. Eso ahorra ancho de banda y hace que el servidor no tenga que manejar archivos pesados.

1.2 MailConfig.java

Para qué sirve

Arma el cliente de **Resend**, que es el servicio por donde salen todos los mails del sistema (verificación de cuenta, recuperación de contraseña, reactivación).

Qué hace por dentro

Método	Qué hace
<code>resend(apiKey)</code>	Devuelve un objeto <code>Resend</code> armado con la API key leída de <code>resend.api-key</code> .

Mismo patrón exacto que `CloudinaryConfig` : una credencial que entra por `@Value` , un objeto que sale como `@Bean` .

Cómo se conecta con el resto

Lo usa `EmailService` . Dato histórico que puede caer en la mesa: originalmente el proyecto usaba **Brevo por SMTP**, pero la cuenta nunca se activó (error `502 SMTP account is not yet activated`) y no hubo forma de resolverlo con soporte. Se migró a **Resend por API REST**. La diferencia técnica: SMTP es un protocolo viejo de correo donde tu servidor "habla" el idioma del correo; una API REST es simplemente hacer una llamada HTTP diciendo "mandá este mail". Es más simple y más confiable. Por eso `EmailService` ya no usa `JavaMailSender` de Spring.

1.3 OpenApiConfig.java

Para qué sirve

Configura **Swagger UI** — la página web autogenerada donde se ven todos los endpoints de la API, con sus parámetros, y se pueden probar desde el navegador sin escribir código.

Qué hace por dentro

Método / atributo	Qué hace
BEARER_SCHEME_NAME (constante "bearerAuth")	El nombre interno del esquema de autenticación.
openApi()	Devuelve el objeto <code>OpenAPI</code> con: el título ("Bajoneá API"), la versión ("1.0"), la descripción, y el esquema de seguridad Bearer/JWT .

La parte importante es el esquema de seguridad: le dice a Swagger "*esta API usa un token JWT en el header*". Gracias a eso, en la página de Swagger aparece el botón "**Authorize**", donde pegás tu token una vez y todas las pruebas posteriores lo mandan solo.

Cómo se conecta con el resto

No se conecta con lógica de negocio, es puramente documentación. Pero sí depende de que `SecurityConfig` tenga las rutas de Swagger como públicas — si no, la propia página de documentación te daría 401.

PARTE 2 — config/security/ (el corazón de la seguridad)

Esta es la parte fuerte de la mesa. Si te preguntan "¿cómo funciona la autenticación en tu proyecto?", la respuesta sale de acá.

La idea general, primero, en una frase

Cuando alguien hace login, el sistema le da un **JWT** (un carnet digital firmado). En cada pedido siguiente el usuario manda ese carnet, y el sistema (a) verifica que la firma sea auténtica, y (b) **además** chequea contra la base de datos que su sesión siga abierta. Recién ahí lo deja pasar, y solo a las rutas que su rol permite.

Ese "además" es la particularidad del proyecto y vale la pena entenderlo, porque un JWT clásico **no** hace eso.

2.1 JwtService.java — el que fabrica y lee el carnet

Para qué sirve

Es el que **genera** el token cuando alguien se loguea, y el que lo **valida y lo abre** cuando alguien lo presenta. Es pura mecánica de JWT, sin reglas de negocio.

Atributos

Atributo	Tipo	Qué es
secret	String	La clave secreta con la que se firman los tokens. Viene de <code>jwt.secret</code> (variable de entorno). Si alguien la roba, puede fabricar tokens falsos.
expirationMs	long	Cuánto dura un token, en milisegundos. En el proyecto: <code>86400000</code> = 24 horas .

Métodos

Método	Qué hace en criollo
<code>generarToken(usuario, sessionId)</code>	Fabrica el carnet. Le mete adentro: el email (<code>sub</code>), el <code>userId</code> , el <code>rol</code> , el <code>sessionId</code> , la fecha de emisión (<code>iat</code>) y la de vencimiento (<code>exp</code>). Después lo firma con la clave secreta.
<code>validarYExtraerClaims(token)</code>	Verifica que la firma sea válida y que no esté vencido. Si está todo bien, devuelve el contenido (los "claims"). Si no, tira excepción.
<code>extraerUserId(claims)</code>	Saca el id del usuario del contenido.
<code>extraerSessionId(claims)</code>	Saca el id de la sesión.
<code>extraerRol(claims)</code>	Saca el rol y lo convierte al enum <code>RolUsuario</code> .
<code>claveFirma()</code> (privado)	Convierte el <code>secret</code> de texto a una clave criptográfica HMAC-SHA usable.

El concepto de "claim" explicado simple

Un JWT es un texto largo con tres partes separadas por puntos: `cabecera.contenido.firma` . El **contenido** son los "claims" — datos sueltos que vos elegís meter. Acá se metieron 3 propios (`userId` , `rol` , `sessionId`) más los estándar (`sub` , `iat` , `exp`).

Ojo con esto, es pregunta típica de mesa: el contenido de un JWT **no está encriptado**, está apenas codificado en Base64 — cualquiera lo puede leer pegándolo en `jwt.io`. Lo que protege el JWT **no es el secreto del contenido, es que no se pueda modificar**: si alguien le cambia el rol de `CLIENTE` a `ADMINISTRADOR` , la firma deja de coincidir y el token se rechaza. Por eso **nunca se mete una contraseña ni datos sensibles adentro de un JWT**.

2.2 AuthenticatedUser.java — la "credencial en mano" del usuario logueado

Para qué sirve

Es un `record` (una clase inmutable súper simple de Java) que representa **al usuario que está haciendo el pedido en este momento**. Se arma en el filtro y queda disponible para todo el resto.

Atributos

Atributo	Tipo	Qué es
<code>userId</code>	<code>Integer</code>	El id del usuario logueado.
<code>sesionId</code>	<code>Integer</code>	El id de su fila en la tabla <code>Sesion</code> .
<code>email</code>	<code>String</code>	Su email.
<code>rol</code>	<code>RolUsuario</code>	Su rol (CLIENTE, DUENO, ADMINISTRADOR...).

Por qué existe / cómo se conecta

Los controllers no reciben el `userId` como parámetro de la URL — **lo sacan de acá**. Eso es importante para la seguridad: si el cliente pudiera mandar su propio `clienteId` en la URL, podría poner el de otro y ver pedidos ajenos. Al sacarlo del token, es imposible falsearlo.

El `sesionId` está adentro específicamente porque `AuthService` (logout, cambio de contraseña) necesita saber **qué fila de sesión cerrar** sin tener que volver a parsear el token.

2.3 JwtAuthenticationFilter.java — el portero de la entrada

Para qué sirve

Es el filtro por el que pasa cada request antes de llegar al controller. Su trabajo es decidir si el que golpea la puerta está autenticado o no.

Extiende `OncePerRequestFilter`, que garantiza que se ejecute **exactamente una vez por pedido** (sin eso, un forward interno podría hacerlo correr dos veces).

Atributos

Atributo	Qué es
<code>jwtService</code>	Para validar y abrir el token.
<code>sesionRepository</code>	Para consultar la tabla <code>Sesion</code> en la base.
<code>authenticationEntryPoint</code>	Para escribir la respuesta de error 401 si algo falla.

El método `doFilterInternal` — la historia paso a paso

- Mira si viene el header `Authorization: Bearer ...`**. Si no viene, no hace nada y deja pasar el pedido (podría ser una ruta pública, como el catálogo). El que decide si esa ruta necesitaba login o no es `SecurityConfig`, más adelante en la cadena.
- Corta el `"Bearer "`** (los primeros 7 caracteres) y se queda con el token pelado.
- Valida la firma** con `jwtService.validarYExtraerClaims`.
- Saca el `sesionId`** y va a la base: `sesionRepository.findById(sesionId)` y pregunta si esa sesión sigue con `activa = true`.
- Si la sesión está cerrada** → limpia el contexto y devuelve **401**, aunque la firma del token fuera perfecta.
- Si está todo bien** → arma el `AuthenticatedUser`, le pone el rol como autoridad (`ROLE_CLIENTE`, `ROLE_DUENO`, `ROLE_ADMINISTRADOR`), lo guarda en el `SecurityContextHolder`, y deja seguir la cadena.
- Si el token es inválido o expiró** (`JwtException`) → 401.

Por qué el paso 4 es la pregunta más probable de la mesa

Un JWT normal es **stateless**: una vez emitido, vale hasta que expire, y **el servidor no lo puede cancelar**. Si a alguien le roban el token, sigue sirviendo 24 horas.

Bajoneá resolvió eso agregando la tabla `Sesion`. Cada login crea una fila con `activa = true`, y el token lleva el `sesionId` adentro. En cada pedido el filtro chequea esa fila. Entonces se puede **invalidar un token de verdad**, poniendo `activa = false`, en 4 casos reales del sistema:

- Logout** — el usuario cierra sesión.
- Bloqueo de cuenta** — 3 intentos fallidos de login.
- Cambio o recuperación de contraseña** — se cierran las sesiones viejas por seguridad.
- Login concurrente** — si entrás de nuevo, la sesión anterior se cae (sesión única por cuenta).

El **trade-off**, por si te lo preguntan: el precio de esto es una consulta a la base **en cada request**, que es exactamente lo que un JWT stateless busca evitar. Se aceptó porque poder cerrar sesiones de verdad vale más que ahorrar esa consulta, en un sistema de este tamaño.

2.4 SecurityConfig.java — el reglamento de quién entra a dónde

Para qué sirve

Es el mapa de permisos de toda la API: qué rutas son públicas, qué rutas pide cada rol, en qué orden corren los filtros, cómo se hashean las contraseñas y qué CORS se permite.

Atributos

Los 4 son componentes que le inyecta Spring: `jwtAuthenticationFilter` , `rateLimitFotoRegistroFilter` , `authenticationEntryPoint` , `accessDeniedHandler` .

RUTAS_PUBLICAS — el array de rutas sin login

Se agrupan así:

- **Auth sin sesión:** registro (cliente y comercio), login, verificar cuenta, reenviar verificación, recuperar contraseña (los 3 pasos), reactivar cuenta (los 2 pasos).
- **Catálogo y geografía:** `/api/v1/catalogo/**` y `/api/v1/geografia/**` — cualquiera puede navegar los comercios y productos, y llenar el selector de provincia/localidad, sin estar logueado.
- **Infraestructura:** `/api/v1/health` , `Swagger` (`/swagger-ui.html` , `/swagger-ui/**` , `/v3/api-docs/**`), `/error` , y `/api/v1/test/**` .

Tres detalles de estas rutas que son historias reales de bugs (buen material para la mesa):

1. `/error` **tiene que ser público**. Si una excepción no la agarra el `GlobalExceptionHandler` , el contenedor hace un forward interno a `/error ...` y ese forward **vuelve a pasar por el filtro de seguridad**. Si `/error` no estaba en la lista, ese segundo paso lo interceptaba `anyRequest().authenticated()` y devolvía **401 en vez del error real** — incluso en un endpoint público. Bug real encontrado probando `GeografiaService` : un parámetro faltante que debía dar 400, daba 401.
2. `/swagger-ui.html` **necesita su propia línea**, aparte de `/swagger-ui/**` . Springdoc lo registra como un **redirect**, no como un subpath, así que el patrón con `**` no lo cubría y la página de docs daba 401.
3. `/api/v1/test/**` **está permitido acá pero no es un agujero de seguridad**. La protección real no es esta lista: `TestController` y `TestSupportService` llevan `@Profile("test")` , así que fuera del perfil de test **Spring ni siquiera crea el bean ni registra la ruta**. Permitirla acá no la hace alcanzable en producción, porque no hay nadie que responda.

securityFilterChain(http) — la cadena de filtros

Es el método central. Lo que configura, en orden:

Configuración	Qué significa en criollo
<code>csrf().disable()</code>	Desactiva la protección CSRF. Se puede porque no usamos cookies de sesión — el token va en un header, y un sitio malicioso no puede leer ni mandar ese header.
<code>cors(...)</code>	Permite que el frontend (que está en otro puerto) pueda llamar al backend.
<code>sessionCreationPolicy(STATELESS)</code>	Spring Security no guarda <code>HttpSession</code> ni cookie propia.
<code>exceptionHandling(...)</code>	Conecta el 401 al <code>authenticationEntryPoint</code> y el 403 al <code>accessDeniedHandler</code> .
<code>authorizeHttpRequests(...)</code>	Las reglas por ruta (abajo).
<code>addFilterBefore(...)</code> x2	Mete los dos filtros propios en la cadena.

Aclaración importante (aparece hasta en el comentario del código): `STATELESS` **no contradice** la tabla `Sesion` . Son dos cosas distintas: `STATELESS` habla de la sesión de servlet de Spring (cookie `JSESSIONID`), que efectivamente no existe; la tabla `Sesion` es estado **de negocio**, propio de la aplicación, validado a mano por el filtro. Si te lo preguntan como si fuera una contradicción, esa es la respuesta.

Las reglas de autorización, en orden

El orden importa muchísimo: **Spring evalúa de arriba hacia abajo y se queda con la primera que matchea**.

#	Ruta	Quién puede
1	<code>RUTAS_PUBLICAS</code>	Cualquiera, sin token.
2	<code>/api/v1/productos/**</code> , <code>/api/v1/comercios/**</code>	Rol DUENO .
3	GET de <code>/api/v1/categorias/**</code> y <code>/api/v1/tags/**</code>	Cualquiera autenticado , sin importar rol.
4	<code>/api/v1/categorias/**</code> , <code>/api/v1/tags/**</code> , <code>/api/v1/administrador/**</code>	Rol ADMINISTRADOR .
5	<code>/api/v1/carrito/**</code> , <code>/api/v1/pedidos/cliente/**</code> , <code>/api/v1/clientes/**</code>	Rol CLIENTE .
6	<code>/api/v1/pedidos/comercio/**</code>	Rol DUENO .
7	Cualquier otra cosa	Autenticado (cualquier rol).

La regla 3 antes que la 4 es intencional y es una buena anécdota: cuando se construyó el formulario de crear producto, el dueño del comercio necesitaba **leer** las categorías para llenar el selector — pero categorías era ADMINISTRADOR-only. Solución: se puso una regla más específica (solo el verbo GET) **antes** que la genérica. Así el comercio puede leer categorías y tags, pero crear/editar/borrar sigue siendo exclusivo del administrador.

Dato para no equivocarte en la mesa: el rol que maneja comercios y productos es **DUENO** , no `COMERCIO` . Eso cambió cuando el proyecto pasó del MVP al modelo completo: ahora un **Dueño** es una persona que puede tener **varios comercios**, así que el rol pasó a estar en la persona, no en el comercio.

El orden de los filtros

RateLimitFotoRegistroFilter → JwtAuthenticationFilter → UsernamePasswordAuthenticationFilter

El rate limit va **primero de todos** — tiene sentido, querés frenar el abuso antes de gastar tiempo validando tokens.

Los otros dos beans

Bean	Qué hace
passwordEncoder()	Devuelve un BCryptPasswordEncoder . Es el que hashea las contraseñas.
corsConfigurationSource()	Permite todos los orígenes (*), los métodos GET/POST/PUT/PATCH/DELETE/OPTIONS, todos los headers, y credenciales.

Sobre BCrypt, por si preguntan: un hash es de **una sola dirección** — de la contraseña sacás el hash, pero del hash no podés volver a la contraseña. Cuando el usuario se loguea, no se "desencripta" nada: se hashea lo que escribió y se compara con lo guardado. Además BCrypt le mete un **salt** aleatorio a cada contraseña, así que dos usuarios con la misma contraseña tienen hashes distintos, y es **lento a propósito**, para que probar millones de contraseñas por fuerza bruta sea inviable.

Sobre el CORS con * : es una configuración de desarrollo, cómoda para que el frontend en localhost:5501 pueda pegarle al backend en localhost:8080 . En producción real convendría restringirlo al dominio propio. *(Vale la pena tenerlo a mano por si preguntan por producción.)*

2.5 CustomAuthenticationEntryPoint.java — el que redacta los 401

Para qué sirve

Cuando alguien intenta entrar **sin estar autenticado** (sin token, con token inválido, vencido, o con sesión cerrada), esta clase escribe la respuesta.

Qué hace

Método	Qué hace
commence(request, response, ex)	Pone status 401 , content-type JSON, UTF-8, y escribe un ApiResponse con el mensaje "No autenticado: token ausente, inválido, expirado o sesión cerrada" .

Por qué existe (esto es lo interesante)

Uno diría "¿y por qué no lo maneja el GlobalExceptionHandler como todos los demás errores?".

Porque **Spring Security intercepta antes**. La cadena de filtros de seguridad corre **antes** del DispatcherServlet (que es el que reparte a los controllers). Si el filtro corta ahí, la petición nunca llega a un controller, y por lo tanto el @RestControllerAdvice nunca se entera. Entonces esta clase escribe la respuesta **directo al HttpServletResponse** , a mano.

Se hace igual para respetar la regla del proyecto de que toda respuesta tenga el mismo formato {mensaje, data} (CLAUDE.md, regla 3). Sin esto, un 401 devolvería el HTML de error por defecto de Spring y el frontend no sabría parsearlo.

2.6 CustomAccessDeniedHandler.java — el que redacta los 403

Para qué sirve

El hermano del anterior, pero para el caso opuesto: el usuario **sí está autenticado**, el token está perfecto, la sesión está viva... pero **su rol no alcanza** para esa ruta.

Qué hace

Método	Qué hace
handle(request, response, ex)	Pone status 403 , JSON, UTF-8, y escribe "No tiene permisos para acceder a este recurso" .

La distinción 401 vs 403 — pregunta clásica de mesa

Código	Significa	Ejemplo en Bajoneá
401 Unauthorized	"No sé quién sos." Falta identificarse.	Pegarle a /api/v1/carrito sin token.
403 Forbidden	"Sé quién sos, pero no podés."	Un CLIENTE logueado pegándole a /api/v1/administrador/comercios .

Truco para no confundirlos: el 401 está mal nombrado en el estándar HTTP — dice "unauthorized" pero en realidad significa "unauthenticated". El 403 es el verdadero "no autorizado".

2.7 RateLimitFotoRegistroFilter.java — el que frena el abuso en el registro

Para qué sirve

Limita cuántas veces por minuto una misma IP puede pedir una firma de Cloudinary durante el registro. Solo aplica a dos rutas:

- `POST /api/v1/auth/registro/comercio/foto-firma`
- `POST /api/v1/auth/registro/cliente/foto-firma`

Por qué hacía falta

Todos los demás endpoints de firma de Cloudinary están protegidos por login — solo un usuario autenticado puede pedirlos, y sabés quién es. Pero **en el registro el usuario todavía no tiene cuenta**, así que esos dos endpoints son necesariamente públicos. Sin ningún freno, cualquiera podría hacer un script que pida firmas infinitas y llene (y facture) la cuenta de Cloudinary.

Atributos

Atributo	Qué es
<code>RUTAS_LIMITADAS</code>	El <code>Set</code> con las 2 rutas afectadas.
<code>limitePorMinuto</code>	Cuántas firmas por minuto por IP. Default 5, configurable vía <code>app.rate-limit.foto-registro-por-minuto</code> .
<code>ventanasPorIp</code>	Un <code>ConcurrentHashMap<String, VentanaRateLimit></code> — la IP como clave, su contador como valor.

Y una clase interna `VentanaRateLimit` con dos campos: `ventana` (qué minuto es) y `contador` (cuántas van en ese minuto).

Cómo funciona el algoritmo, simple

Se llama "ventana fija":

1. Si la ruta no es una de las dos, o es un `OPTIONS` (el preflight de CORS), pasa de largo.
2. Agarra la IP y calcula en qué minuto estamos: `Instant.now().getEpochSecond() / 60`.
3. Busca (o crea) el contador de esa IP.
4. Si el minuto cambió respecto de la última vez → resetea el contador a 0.
5. Suma 1.
6. Si pasó el límite → devuelve **429 Too Many Requests** con un JSON en el mismo formato `{mensaje, data}` del resto de la API.

Las limitaciones, dichas de frente (buen punto para mostrar criterio)

- **Está en memoria.** Si el servidor se reinicia, los contadores se van a cero.
- **Sirve para un solo servidor.** Si mañana hubiera dos instancias, cada una tendría su propio contador y el límite real sería el doble. Para eso haría falta algo compartido, tipo Redis. El propio comentario del código lo dice.
- **Ventana fija tiene un borde conocido:** podés meter 5 en el segundo 59 y 5 más en el 61, o sea 10 en dos segundos. Un algoritmo de ventana deslizante lo evitaría, pero para este caso no vale la complejidad.

Por qué el límite es configurable (anécdota real)

La suite de tests E2E de Playwright registra varios comercios reales en pocos segundos, todos desde la misma IP (localhost). Con 5/min saltaba un **429 del propio backend**, no de Cloudinary — los tests fallaban por el rate limit, no por un bug. Solución: se dejó el número configurable, así `application-test.properties` lo sube sin tocar el comportamiento real de producción.

Resumen visual: el recorrido de un request



Preguntas típicas de mesa sobre esta carpeta (con respuesta corta)

"¿Cómo autenticás usuarios?" Con JWT. En el login se valida usuario y contraseña, se crea una fila en `Sesion` y se emite un token firmado que lleva adentro el email, el `userId`, el rol y el `sesionId`. Cada request posterior manda ese token en el header `Authorization: Bearer`.

5/9/26, 6:02 p.m.

Markdown Live Preview

"¿Qué pasa si le roban el token a alguien?" Como el token lleva un `sesionId` y en cada pedido se chequea que esa sesión siga activa, se puede invalidar poniendo `activa = false` . Un JWT puramente stateless no permitiría eso.

"¿Dónde guardás las contraseñas?" Hasheadas con BCrypt, nunca en texto plano. El hash es de una vía y lleva salt propio por contraseña.

"¿Por qué desactivaste CSRF?" Porque no uso cookies de sesión. El token viaja en un header, y un sitio de terceros no puede inyectar ese header en una petición del usuario.

"¿Qué diferencia hay entre 401 y 403 en tu sistema?" 401 lo escribe `CustomAuthenticationEntryPoint` : no sé quién sos. 403 lo escribe `CustomAccessDeniedHandler` : sé quién sos pero tu rol no alcanza.

"¿Cómo hacés que el rol viaje?" Va como claim dentro del JWT. El filtro lo lee y lo carga como `ROLE_X` en el contexto de Spring Security, que es lo que después evalúan las reglas de `hasRole(...)` .

Índice de archivos cubiertos en este documento

Archivo	Qué es, en una línea
config/CloudinaryConfig.java	Bean del cliente de Cloudinary con las credenciales.
config/MailConfig.java	Bean del cliente de Resend para mandar mails.
config/OpenApiConfig.java	Config de Swagger UI, con el esquema Bearer/JWT.
config/security/AuthenticatedUser.java	Record con los datos del usuario logueado (userId, sessionId, email, rol).
config/security/JwtService.java	Genera, valida y parsea el JWT.
config/security/JwtAuthenticationFilter.java	Filtro que valida token + sesión activa en cada request.
config/security/SecurityConfig.java	Reglas de qué rol accede a qué ruta, filtros, BCrypt, CORS.
config/security/CustomAuthenticationEntryPoint.java	Escribe la respuesta 401 con el formato estándar.
config/security/CustomAccessDeniedHandler.java	Escribe la respuesta 403 con el formato estándar.
config/security/RatelimitFotoRegistroFilter.java	Limita 5 firmas de Cloudinary por minuto por IP en el registro.